Análise Empírica de Algoritmos de Ordenação: Uma Avaliação Experimental de Bubble Sort, Selection Sort, Insertion Sort e Shell Sort em Entradas de Grande Volume

Gustavo H. Mazur[1]

Departamento de Informática – Universidade Tecnológica Federal do Paraná (UTFPR)
gustavomazur7475@gmail.com

Abstract

Sorting algorithms are among the most fundamental structures in Computer Science, with well-established theoretical complexities. However, asymptotic analysis abstracts hardware constants, memory effects, and input distribution, which can cause significant divergences between theoretical predictions and observed behavior. This work presents an empirical study comparing four classical sorting algorithms — Bubble Sort, Selection Sort, Insertion Sort, and Shell Sort — through a reproducible benchmarking infrastructure implemented in Go 1.23. Experiments were conducted on five input sizes (175,000 to 1,500,000 elements) under three dataset configurations (sorted, reversed, and random), with three independent runs per combination. Metrics collected include execution time, elementary operations (comparisons and swaps), and heap memory allocations. The results empirically confirm theoretical complexity predictions and reveal relevant practical distinctions: Bubble Sort and Insertion Sort achieve near-zero execution time for sorted inputs; Selection Sort maintains constant behavior regardless of input order; and Shell Sort outperforms all $O(n^2)$ algorithms by up to 5213 times for random inputs of 750,000 elements, demonstrating that complexity class membership is the dominant factor in large-scale performance.
Keywords: Sorting Algorithms, Empirical Analysis, Computational Complexity, Go Language.

Resumo

Algoritmos de ordenação figuram entre as estruturas mais fundamentais da Ciência da Computação, com complexidades teóricas amplamente estabelecidas. Contudo, a análise assintótica abstrai constantes de hardware, efeitos de memória e distribuição de entrada, o que pode gerar divergências significativas entre predições teóricas e comportamento observado. O presente trabalho apresenta um estudo empírico comparando quatro algoritmos de ordenação clássicos — Bubble Sort, Selection Sort, Insertion Sort e Shell Sort — por meio de uma infraestrutura de benchmarking reprodutível implementada em Go 1.23. Os experimentos foram conduzidos sobre cinco tamanhos de entrada (175.000 a 1.500.000 elementos) sob três configurações de dataset (ordenado, invertido e aleatório), com três execuções independentes por combinação. As métricas coletadas incluem tempo de execução, operações elementares (comparações e trocas) e alocações de memória em *heap*. Os resultados confirmam empiricamente as predições de complexidade teórica e revelam distinções práticas relevantes:

Bubble Sort e Insertion Sort atingem tempo de execução praticamente nulo para entradas ordenadas; Selection Sort mantém comportamento constante independentemente da ordem da entrada; e Shell Sort supera os demais algoritmos em até 5213 vezes para entradas aleatórias com 750.000 elementos, demonstrando que a classe de complexidade é o fator dominante no desempenho em grande escala.
Palavras-chave: Algoritmos de Ordenação, Análise Empírica, Complexidade Computacional, Linguagem Go.

1 Introdução

A ordenação de dados constitui um dos problemas centrais da Ciência da Computação. Algoritmos de ordenação estão presentes em praticamente todas as camadas de sistemas de software, desde rotinas de banco de dados até renderização gráfica e indexação de mecanismos de busca [2]. A importância desse problema motivou décadas de pesquisa, resultando em algoritmos com complexidades assintóticas bem conhecidas e provadas.

A análise de complexidade assintótica, expressa em notação $O(\cdot)$, descreve o comportamento de um algoritmo no limite, quando o tamanho da entrada n tende ao infinito. Essa abstração é teoricamente rigorosa, porém deliberadamente ignora constantes multiplicativas, coeficientes de ordem inferior e efeitos de hardware, tais como localidade de *cache*, *pipeline* de processador e custo de alocação de memória [3]. Em contextos práticos, esses fatores podem produzir diferenças de desempenho substanciais entre algoritmos pertencentes à mesma classe de complexidade.

Diante desse cenário, o presente trabalho adota uma abordagem complementar à análise teórica: a avaliação empírica e reprodutível do desempenho de algoritmos de ordenação em condições reais de hardware. Especificamente, este estudo tem os seguintes objetivos:

1. Implementar e instrumentar quatro algoritmos clássicos de ordenação com contagem exata de comparações e trocas;
2. Executar os algoritmos sobre três tipos de dataset (ordenado, invertido e aleatório) em cinco tamanhos de entrada (175.000 a 1.500.000 elementos);
3. Comparar os resultados empíricos com as predições teóricas de complexidade;
4. Identificar e discutir divergências e nuances práticas não capturadas pela análise assintótica.

Os algoritmos avaliados são: Bubble Sort, Selection Sort, Insertion Sort e Shell Sort. Esses quatro algoritmos foram selecionados por representarem uma progressão de complexidade e estratégias distintas de ordenação, permitindo uma análise comparativa abrangente dentro da família de algoritmos baseados em comparação.

O restante deste artigo está organizado da seguinte forma: a Seção 2 apresenta o referencial teórico dos algoritmos avaliados; a Seção 3 discute os trabalhos relacionados; a Seção 4 descreve a metodologia experimental adotada; a Seção 5 apresenta os resultados obtidos; a Seção 6 analisa e discute esses resultados; e a Seção 7 conclui o artigo.

2 Referencial Teórico

2.1 Bubble Sort

O Bubble Sort é um dos algoritmos de ordenação mais simples e didaticamente reconhecidos. Seu funcionamento baseia-se em passagens sucessivas sobre o vetor, comparando e trocando pares de elementos adjacentes quando encontrados fora de ordem [2]. A cada passagem completa, o maior elemento não posicionado é propagado (”borbulhado”) ao final do vetor.

A complexidade de tempo do Bubble Sort é $O(n^2)$ no caso médio e no pior caso, e $O(n)$ no melhor caso, quando o vetor já se encontra ordenado — condição detectável por uma *flag* de troca [2]. No pior caso, para um vetor de n elementos em ordem invertida, a implementação não otimizada realiza $n(n-1)$ comparações, pois cada passagem percorre o vetor inteiro sem redução do limite superior. A complexidade de espaço é $O(1)$, pois o algoritmo opera *in-place*.

2.2 Selection Sort

O Selection Sort divide conceitualmente o vetor em duas partes: uma porção ordenada (à esquerda) e uma não ordenada (à direita). A cada iteração, o algoritmo seleciona o menor elemento da porção não ordenada e o posiciona ao final da porção ordenada, mediante uma única troca [2].

Uma característica fundamental do Selection Sort é a ausência de melhor caso em termos de comparações: independentemente da distribuição de entrada, o algoritmo realiza exatamente $\frac{n(n-1)}{2}$ comparações, o que implica complexidade $O(n^2)$ para todos os casos. Em contrapartida, o número máximo de trocas é $n-1$ — propriedade vantajosa em cenários onde a operação de escrita é significativamente mais custosa que a de leitura [3]. A complexidade de espaço é $O(1)$.

2.3 Insertion Sort

O Insertion Sort constrói o vetor ordenado incrementalmente: a cada iteração, o algoritmo insere o elemento i-ésimo na posição correta dentro dos $i-1$ elementos já ordenados, deslocando os demais elementos à direita [2].

A complexidade de tempo é $O(n^2)$ no pior caso (vetor invertido) e $O(n)$ no melhor

caso (vetor já ordenado), pois neste último cada elemento requer exatamente uma comparação para confirmar sua posição. O Insertion Sort é particularmente eficiente para entradas pequenas ou parcialmente ordenadas, sendo o algoritmo de escolha para finalização em implementações híbridas modernas, como o Tim Sort, utilizado em Python e Java [2]. A complexidade de espaço é $O(1)$.

2.4 Shell Sort

O Shell Sort, proposto por Donald Shell em 1959, estende o Insertion Sort introduzindo o conceito de *gap*: em vez de comparar apenas elementos adjacentes, o algoritmo compara e ordena elementos separados por um intervalo h, reduzindo progressivamente esse intervalo até $h = 1$ [6]. Essa estratégia permite mover elementos distantes de sua posição correta em poucas operações, reduzindo o número total de trocas.

A complexidade do Shell Sort depende da sequência de gaps utilizada. Com a sequência original de Shell ($h = \lfloor n/2 \rfloor, \lfloor n/4 \rfloor, \ldots, 1$), a complexidade de pior caso é $O(n^2)$, mas o comportamento prático é significativamente melhor, observado empiricamente como $O(n \log^2 n)$ em média [3]. Sequências de gaps mais sofisticadas, como as de Hibbard e Sedgewick, garantem complexidade $O(n^{3/2})$ e $O(n^{4/3})$ no pior caso, respectivamente [5]. A complexidade de espaço é $O(1)$.

3 Trabalhos Relacionados

A literatura sobre análise experimental de algoritmos de ordenação pode ser organizada em dois grupos principais: estudos empíricos comparativos de desempenho e análises específicas de algoritmos e sua interação com características de hardware.

Estudos Empíricos Comparativos de Desempenho

Pizarro-Vasquez et al. [4] conduziram uma avaliação empírica de onze algoritmos de ordenação — incluindo Bubble Sort, Insertion Sort, Selection Sort, Shell Sort, Merge Sort, Heap Sort e Quick Sort — implementados em C e Python. Os experimentos utilizaram quatro configurações de entrada (aleatória, quase ordenada, invertida e com poucos valores únicos), avaliando tempos de execução. Os autores confirmaram que algoritmos de classe $O(n \log n)$ superam consistentemente os de classe $O(n^2)$ à medida que o tamanho da entrada cresce. Contudo, o estudo não instrumenta contagens exatas de operações elementares nem quantifica a variabilidade estatística entre execuções independentes, aspectos que o presente trabalho aborda explicitamente.

Biggar et al. [1] investigaram o comportamento de algoritmos de ordenação clássicos sob a perspectiva de predição de desvios (*branch prediction*) em processadores modernos. Os autores demonstraram que algoritmos com mesma complexidade assintótica podem divergir

substancialmente em desempenho prático em função de características microarquiteturais, como *miss rate* do preditor de desvios e localidade de *cache*. Esse estudo evidencia que fatores de hardware — abstraídos pela notação $O(\cdot)$ — são determinantes no desempenho observado em condições reais, hipótese que também motiva a abordagem experimental adotada neste trabalho.

Análise Teórico-Empírica do Shell Sort

Sedgewick [5] apresentou uma análise abrangente do Shell Sort e das sequências de gaps que determinam sua complexidade. O trabalho demonstrou que a escolha da sequência é determinante: a sequência original de Shell resulta em $O(n^2)$ no pior caso, enquanto as sequências de Hibbard e Sedgewick garantem $O(n^{3/2})$ e $O(n^{4/3})$, respectivamente. A análise fornece embasamento teórico para a interpretação do comportamento empírico do Shell Sort investigado neste artigo.

Diferentemente dos trabalhos acima, o presente artigo adota instrumentação *in-code* que contabiliza exatamente as comparações e trocas realizadas por cada algoritmo, em vez de apenas medir tempo de execução. A infraestrutura de benchmarking é implementada em Go — linguagem que compila para código de máquina nativo, sem *overhead* de JIT ou interpretação — e avalia entradas de até 1.500.000 elementos com três execuções independentes por combinação, permitindo quantificar a variabilidade das medições. Essa combinação oferece uma análise empírica mais granular e reprodutível do que os estudos identificados na literatura.

4 Metodologia

4.1 Ambiente Experimental

Todos os experimentos foram executados em hardware dedicado, sem virtualização, conforme descrito na Tabela 1. O sistema operacional não foi reiniciado entre as execuções, mas o benchmarking foi isolado por invocações explícitas do Garbage Collector antes de cada medição, conforme detalhado na Seção 4.2.

4.2 Implementação e Instrumentação

A linguagem Go foi escolhida por atender a requisitos científicos precisos para benchmarking de algoritmos. Por ser compilada estaticamente para código de máquina nativo (x86-64), Go elimina o *overhead* de interpretação e o *warm-up* de compiladores JIT, presentes em linguagens como Java e C#. O tempo medido reflete, portanto, o custo computacional real das operações elementares.

Table 1. Especificações do ambiente de experimentação.

Componente	Especificação
Modelo	Dell G15 5515
Processador	AMD Ryzen 5 5600H, 3,3 GHz base, 6 núcleos / 12 threads
Arquitetura	x86-64
Memória RAM	16 GB DDR4
Sistema Operacional	Windows 11 Home (Build 26200)
Linguagem	Go 1.23 (compilação nativa, sem JIT)

Cada algoritmo foi implementado como uma estrutura concreta satisfazendo a interface `SortAlgorithm`, que exige dois métodos: `Name() string` e `Sort(arr []int) SortResult`. A estrutura `SortResult` encapsula o vetor ordenado, o total de comparações e o total de trocas, contados incrementalmente durante a execução — instrumentação *in-code* sem necessidade de interceptação externa.

O motor de benchmarking (`runner.go`) adota as seguintes garantias de isolamento:

- **Cópia limpa de dados:** cada execução recebe uma cópia independente do dataset original via `data.Clone`, evitando que a ordenação in-place de uma execução afete as subsequentes.
- **Limpeza do Garbage Collector:** duas chamadas explícitas a `runtime.GC()` antes de cada medição eliminam o ruído causado por coletas de ciclos anteriores.
- **Medição de tempo de alta resolução:** `time.Since(start)` utiliza o relógio monotônico do sistema operacional com resolução de nanossegundos, sem operações de I/O entre as marcas de tempo.
- **Medição de memória:** `runtime.MemStats` é capturado antes e após cada ordenação, contabilizando apenas as alocações de *heap* ocorridas durante a execução.

A reprodutibilidade dos datasets aleatórios é garantida por semente fixa no gerador pseudoaleatório (`rand.NewSource(42)`), assegurando que todos os experimentos recebam exatamente os mesmos dados de entrada em qualquer execução ou máquina.

4.3 Design do Experimento

Cada combinação (algoritmo $\times$ tipo de dataset $\times$ tamanho de entrada) foi executada três vezes de forma independente. Os tipos de dataset e sua correspondência com os casos teóricos são apresentados na Tabela 2.

Os tamanhos de entrada avaliados foram: 175.000, 250.000, 500.000, 750.000 e 1.500.000 elementos. Para cada grupo (algoritmo $\times$ dataset), foram calculadas as seguintes

Table 2. Tipos de dataset e correspondência com casos teóricos.

Dataset	Descrição	Caso teórico
`ordenado`	$[1, 2, 3, \ldots, n]$	Melhor caso (Bubble, Insertion)
`invertido`	$[n, n{-}1, \ldots, 1]$	Pior caso (Bubble, Insertion)
`aleatório`	Permutação com seed fixa 42	Caso médio

estatísticas a partir das três execuções: mínimo, máximo, média aritmética, desvio padrão e mediana.

4.4 Métricas Coletadas

As métricas registradas por execução foram: (i) duração em nanossegundos e milissegundos; (ii) número de comparações elemento-a-elemento; (iii) número de trocas ou deslocamentos; e (iv) alocações de heap em kilobytes. Os resultados foram exportados para arquivos CSV e processados com a biblioteca pandas em Python para geração de gráficos (matplotlib/seaborn) e tabelas estatísticas.

5 Resultados

5.1 Operações Elementares

A Tabela 3 apresenta o número de comparações e trocas realizadas por cada algoritmo para $n = 750.000$ elementos. Os valores são idênticos nas três execuções de um mesmo grupo, pois a contagem de operações elementares é determinística para uma dada combinação de algoritmo e dataset.

5.2 Tempo de Execução

A Tabela 4 apresenta as estatísticas de tempo de execução para $n = 750.000$ elementos, incluindo média e desvio padrão das três execuções independentes.

As Figuras 1 e 2 apresentam os tempos médios de execução em escala logarítmica (base 10) para entradas de 175.000 e 750.000 elementos, respectivamente. A escala logarítmica é necessária para a visualização simultânea de algoritmos cujos tempos diferem em até quatro ordens de grandeza.

Table 3. Operações elementares para $n = 750.000$ elementos.

Algoritmo	Dataset	Comparações	Trocas / Deslocamentos
Bubble Sort	ordenado	749999	0
Bubble Sort	invertido	562499250000	281249625000
Bubble Sort	aleatório	562330500225	141144955556
Selection Sort	ordenado	281249625000	0
Selection Sort	invertido	281249625000	375000
Selection Sort	aleatório	281249625000	748950
Insertion Sort	ordenado	749999	0
Insertion Sort	invertido	281249625000	281249625000
Insertion Sort	aleatório	141074811900	141074811900
Shell Sort	ordenado	13550103	0
Shell Sort	invertido	19813068	7059999
Shell Sort	aleatório	46534412	33386654

6 Análise e Discussão

6.1 Confirmação Empírica das Complexidades Teóricas

Os resultados obtidos confirmam com precisão as predições teóricas de complexidade. O número de comparações do Selection Sort foi exatamente $\frac{n(n-1)}{2} = 281249625000$ para os três tipos de dataset, corroborando analiticamente a invariante do algoritmo: o laço interno percorre sempre os $n - i$ elementos restantes da porção não ordenada, independentemente da distribuição de entrada.

Para o Bubble Sort com vetor invertido, o número de comparações observado foi $n(n-1) = 562499250000$. Esse valor, superior ao $\frac{n(n-1)}{2}$ usual, decorre de uma característica da implementação: o limite do laço interno não é reduzido a cada passagem, de modo que todas as n passagens percorrem os $n - 1$ pares adjacentes. Para entradas já ordenadas, a *flag* `swapped` encerra o algoritmo após uma única passagem de $n - 1$ comparações — confirmado pelo resultado de 749999 comparações e tempo $\approx$ 0 ms observados experimentalmente.

O Shell Sort com sequência de gaps $\lfloor n/2 \rfloor, \lfloor n/4 \rfloor, \ldots, 1$ realizou 46534412 comparações para entrada aleatória com $n = 750.000$ — menos de $0,01\%$ das 562499250000 comparações do Bubble Sort no mesmo cenário. Esse resultado valida empiricamente o comportamento sub-quadrático observado na prática.

Table 4. Tempo de execução para $n = 750.000$ elementos (em ms).

Algoritmo	Dataset	Mín.	Máx.	Média	Desv. Pad.
Bubble Sort	ordenado	0,000	0,000	0,000	0,000
Bubble Sort	invertido	405.078	406.336	405.693	514
Bubble Sort	aleatório	771.260	797.288	785.339	10.732
Selection Sort	ordenado	285.350	287.238	286.033	854
Selection Sort	invertido	250.354	251.665	250.840	586
Selection Sort	aleatório	262.922	294.105	279.641	12.829
Insertion Sort	ordenado	0,000	0,000	0,000	0,000
Insertion Sort	invertido	168.444	197.964	183.619	12.065
Insertion Sort	aleatório	83.476	85.616	84.355	914
Shell Sort	ordenado	16	22	19	2
Shell Sort	invertido	24	25	25	0,673
Shell Sort	aleatório	139	159	150	8

6.2 Melhor Caso: Divergências entre Algoritmos O(n²)

Um dos achados mais expressivos deste estudo é a diferença de comportamento no melhor caso entre algoritmos de mesma classe assintótica. Bubble Sort e Insertion Sort atingiram tempo de execução estatisticamente nulo (< 1 ms) para vetores já ordenados com $n = 750.000$, realizando exatamente $n - 1 = 749999$ comparações — o mínimo possível para qualquer algoritmo baseado em comparação que verifica a ordenação.

O Selection Sort, por sua vez, consumiu 286033 ms ($\approx 4,8$ minutos) para o mesmo dataset, realizando as mesmas $\frac{n(n-1)}{2}$ comparações de seus casos invertido e aleatório. Esse comportamento evidencia uma limitação estrutural do algoritmo: a busca pelo mínimo da porção não ordenada não pode ser encurtada, mesmo que os dados já estejam em ordem.

A prática consequência desse resultado é que a notação $O(n^2)$, compartilhada pelos três algoritmos, é uma descrição insuficiente para a tomada de decisão em aplicações reais. A distribuição da entrada é um fator crítico que a análise assintótica não captura.

6.3 Divergência Prática entre Algoritmos de Mesma Complexidade

Bubble Sort e Insertion Sort compartilham a complexidade $O(n^2)$ no caso médio, mas apresentaram tempos de execução substancialmente distintos para entradas aleatórias. Para $n = 750.000$, o Insertion Sort executou em 84355 ms em média, enquanto o Bubble Sort consumiu 785339 ms — uma diferença de fator $\approx 9{,}3\times$. Essa divergência tem explicação estrutural: o Insertion Sort interrompe o laço interno assim que encontra um elemento menor ou igual à chave sendo inserida, realizando no caso aleatório aproximadamente $\frac{n(n-1)}{4}$ comparações; o Bubble Sort, na implementação avaliada, não reduz o limite do laço interno,

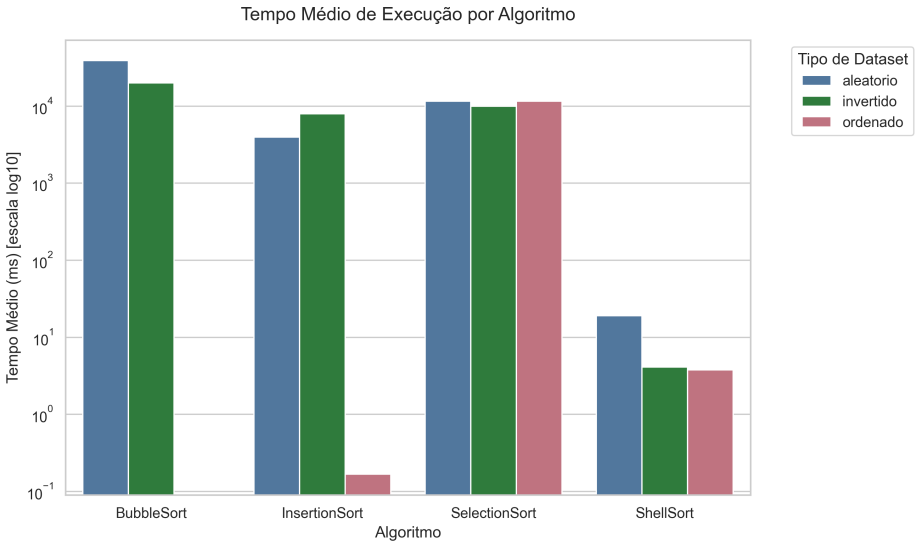

Figure 1. Tempo médio de execução para $n = 175.000$ elementos (escala $\log_{10}$).

percorrendo o vetor completo em cada passagem.

No caso invertido, o Insertion Sort (183.619 ms) permanece mais rápido que o Bubble Sort (405.693 ms) — fator $\approx 2{,}2\times$ —, porém com vantagem substancialmente menor do que no caso aleatório (9,3×). A explicação está na mudança do padrão de operações: no pior caso (entrada invertida), o Insertion Sort realiza exatamente $\frac{n(n-1)}{2}$ comparações e $\frac{n(n-1)}{2}$ deslocamentos; no caso aleatório, interrompe o laço interno precocemente, executando apenas $\approx \frac{n(n-1)}{4}$ operações. O Bubble Sort, por sua vez, realiza $n(n-1)$ comparações em ambos os casos — o que explica por que a vantagem relativa do Insertion Sort é maior no caso aleatório. Ambos confirmam a complexidade $O(n^2)$ no pior caso, e esses resultados demonstram que constantes multiplicativas ocultas pela notação assintótica têm impacto mensurável no desempenho prático.

6.4 Shell Sort: Diferença de Classe de Complexidade

O Shell Sort evidenciou, de forma inequívoca, o impacto prático da classe de complexidade. Para $n = 750.000$ com entrada aleatória, o algoritmo executou em 150,618 ms em média, contra 785339,856 ms do Bubble Sort — uma diferença de fator $\approx 5213\times$. Para $n = 1.500.000$, essa diferença supera $8000\times$, evidenciando o crescimento divergente das complexidades $O(n^2)$ e $O(n \log^2 n)$ à medida que n cresce.

Nota-se ainda que o Shell Sort apresentou tempo de execução inferior para entradas invertidas (25,118 ms) em comparação com entradas aleatórias (150,618 ms). Esse comportamento é atribuível à estrutura dos gaps: vetores invertidos possuem padrão regular que o Shell Sort explora eficientemente nas primeiras passagens com gaps grandes, posicionando elementos próximos de sua posição final com menor custo nas passagens

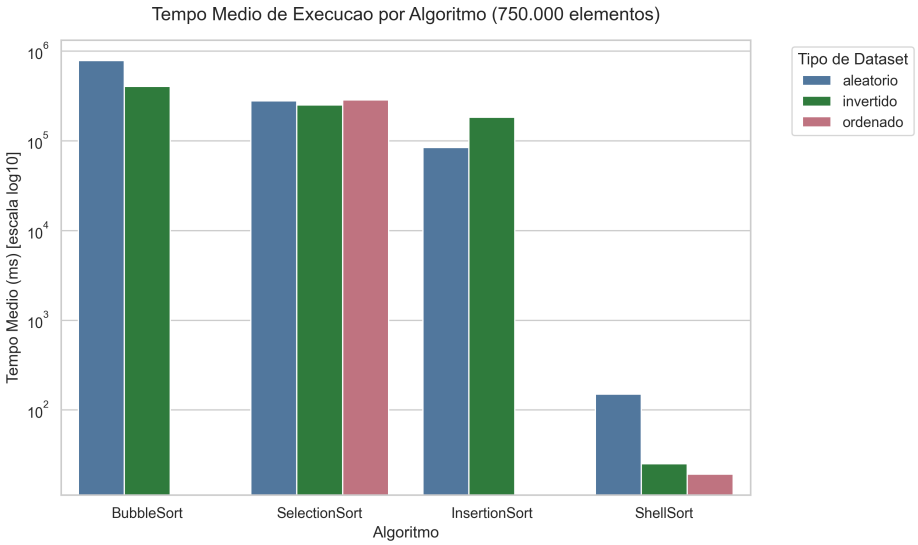

Figure 2. Tempo médio de execução para $n = 750.000$ elementos (escala $\log_{10}$).

subsequentes.

6.5 Estabilidade das Medições

O desvio padrão das medições de tempo oferece uma estimativa da variabilidade introduzida pelo ambiente de execução (interferências do sistema operacional, cache miss, etc.). Para o Bubble Sort com entrada aleatória, o desvio padrão foi de 10732 ms ($1,37\%$ da média), valor aceitável dado o longo tempo de execução. O Shell Sort apresentou variabilidade relativa maior em termos percentuais para o caso ordenado ($12,77\%$), o que é esperado para medições de tempo absoluto reduzido, onde variações de poucos microssegundos representam fração perceptível.

Todos os algoritmos apresentaram alocações de heap nulas (0 KB), confirmando o comportamento in-place teórico: nenhum dos quatro algoritmos avaliados utiliza estruturas de dados auxiliares proporcionais a n.

7 Conclusão

Este trabalho apresentou uma avaliação empírica e reprodutível de quatro algoritmos clássicos de ordenação. Os experimentos conduzidos sobre entradas de até 1.500.000 elementos confirmaram quantitativamente as predições da análise assintótica e revelaram nuances práticas relevantes que a notação $O(\cdot)$ não captura.

Os principais achados podem ser sintetizados em três constatações. Em primeiro lugar, a distribuição da entrada exerce impacto determinante no desempenho de algoritmos

que possuem melhor caso diferente do pior caso: Bubble Sort e Insertion Sort exibiram tempo de execução praticamente nulo para vetores ordenados, enquanto Selection Sort permaneceu invariante. Em segundo lugar, algoritmos de mesma classe de complexidade podem apresentar diferenças práticas de desempenho de até $9{,}3\times$ para o mesmo tamanho de entrada, decorrentes de constantes multiplicativas distintas. Em terceiro lugar, a diferença de classe de complexidade é o fator dominante em grande escala: Shell Sort superou todos os algoritmos $O(n^2)$ em até 8000 vezes para as maiores entradas avaliadas.

Como trabalhos futuros, planeja-se incorporar algoritmos de classe $O(n \log n)$ — Merge Sort, Quick Sort e Heap Sort — bem como algoritmos não comparativos — Counting Sort e Radix Sort — à infraestrutura de benchmarking. A análise estatística formal, com testes de hipótese para comparação de médias entre execuções, e a geração de curvas de crescimento temporal (tempo $\times$ n) para ajuste de curva são também etapas previstas para consolidação dos resultados experimentais.

References

[1] Paul Biggar, Nicholas Nash, Kevin Williams, and David Gregg. An experimental study of sorting and branch prediction. *ACM Journal of Experimental Algorithmics*, 12:1.8:1–1.8:39, 2008.

[2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, Cambridge, 3 edition, 2009.

[3] Donald E. Knuth. *The Art of Computer Programming, Volume 3: Sorting and Searching*. Addison-Wesley, Reading, 2 edition, 1998.

[4] Galo Octavio Pizarro-Vasquez, Franklin Mejia Morales, Pietro Galvez Minervini, and Miguel Botto-Tobar. Sorting algorithms and their execution times: An empirical evaluation. In *Technology Trends — ICAETT 2020*, volume 1302 of *Advances in Intelligent Systems and Computing*, Cham, 2020. Springer.

[5] Robert Sedgewick. Analysis of shellsort and related algorithms. In *Algorithms — ESA '96, 4th Annual European Symposium*, volume 1136 of *Lecture Notes in Computer Science*, Berlin, Heidelberg, 1996. Springer.

[6] Donald L. Shell. A high-speed sorting procedure. *Communications of the ACM*, 2(7):30–32, July 1959.